

SLEEP STAGE CLASSIFICATION

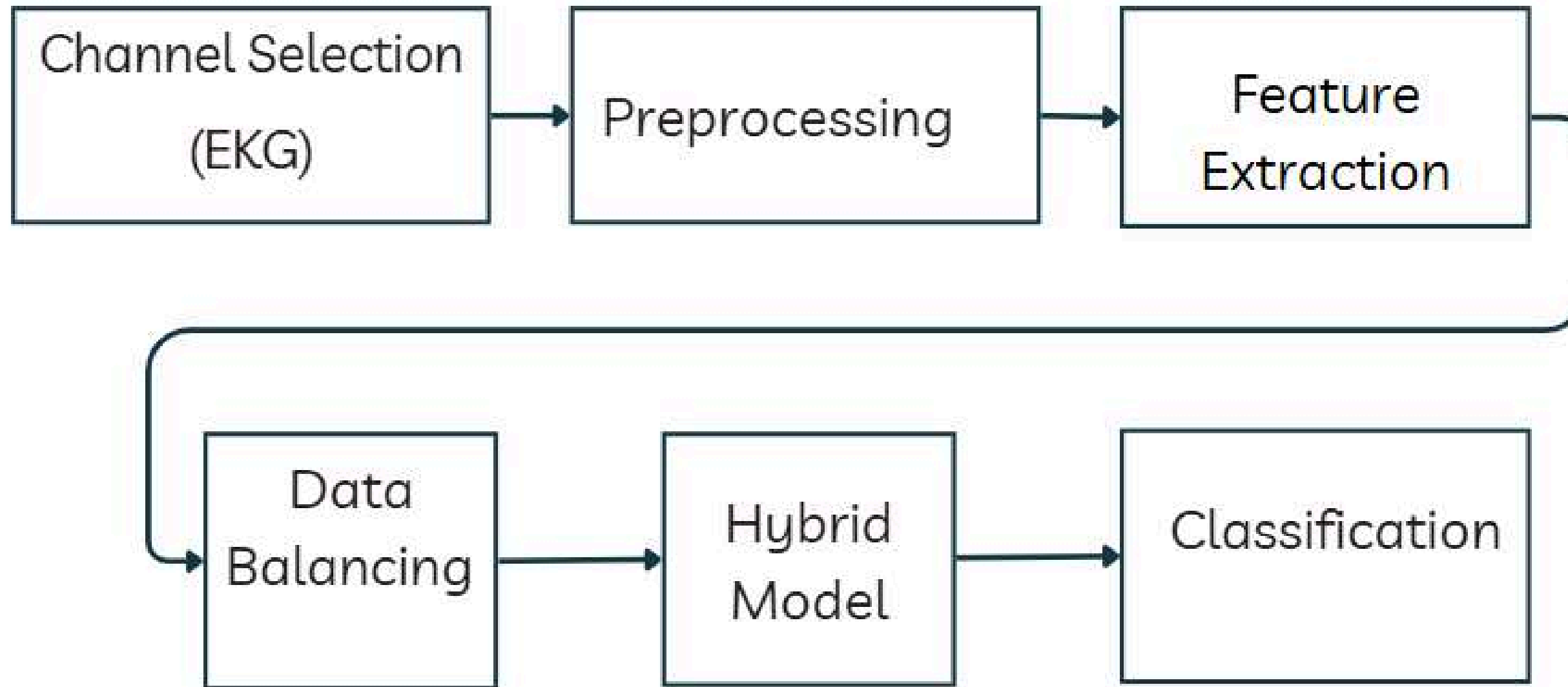
Using ECG-Based IHR Signals

By: Zeinab Abdo

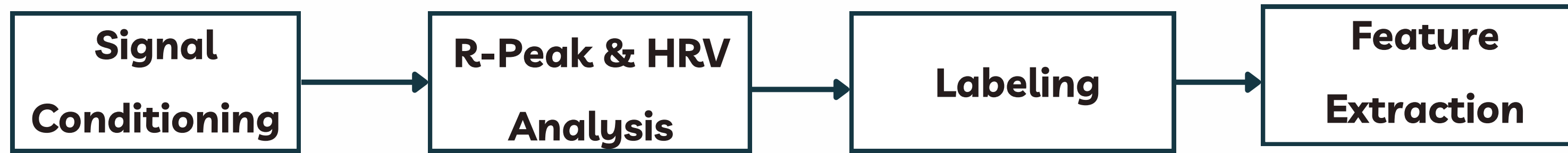
Supervised by: Dr. Ibrahim Sadek | Eng. Veronica William



PIPELINE

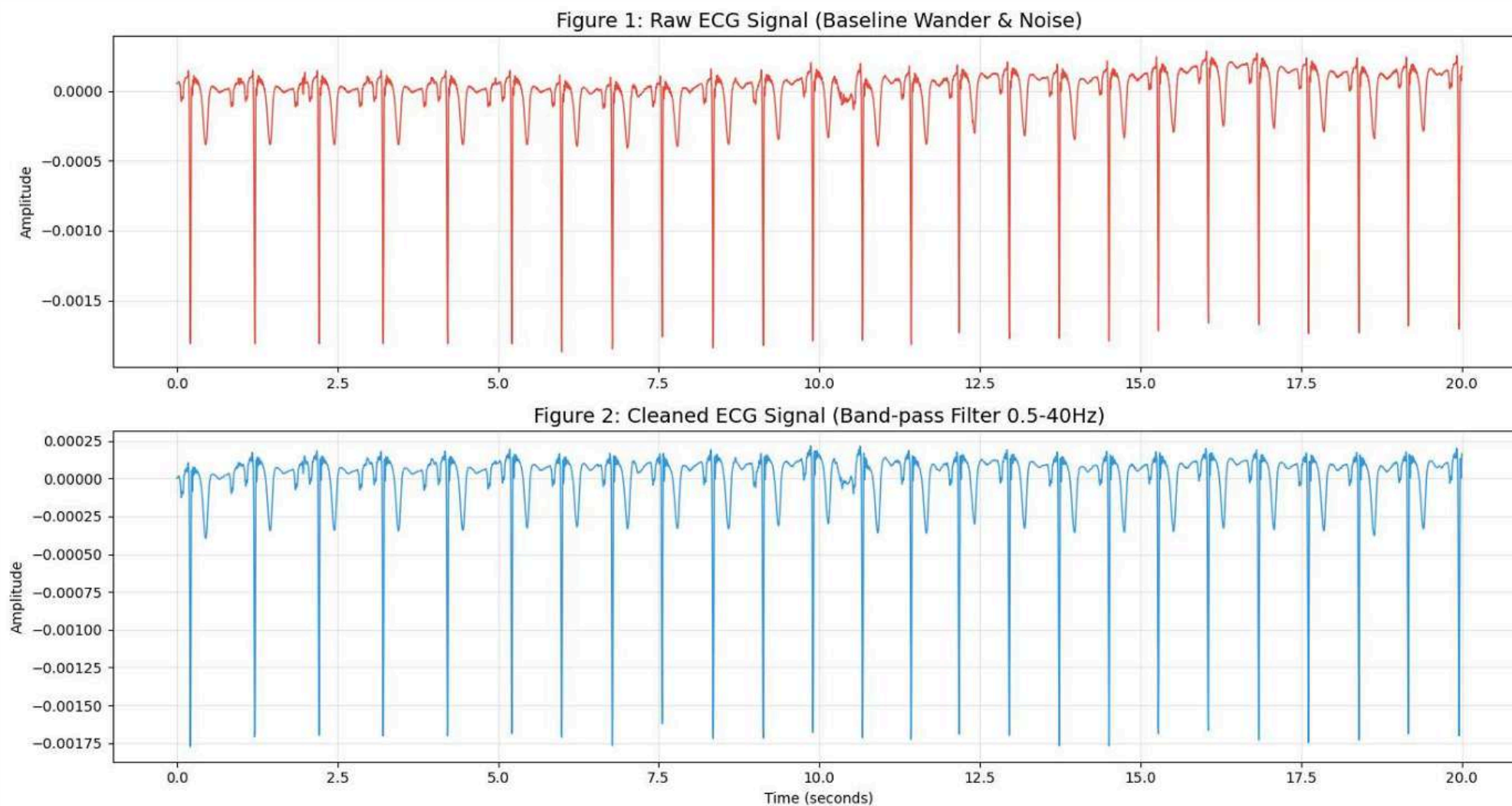


DATA PROCESSING PIPELINE



1.SIGNAL LOADING AND CONDITIONING

- `mne.io.read_raw_edf`: For reading medical-grade EDF data.
- `raw.pick(['EKG'])`: To focus exclusively on the electrocardiogram signal.
- `raw.filter(0.5, 40.0)`: Applied a Bandpass Filter to eliminate baseline wander and high-frequency powerline noise.



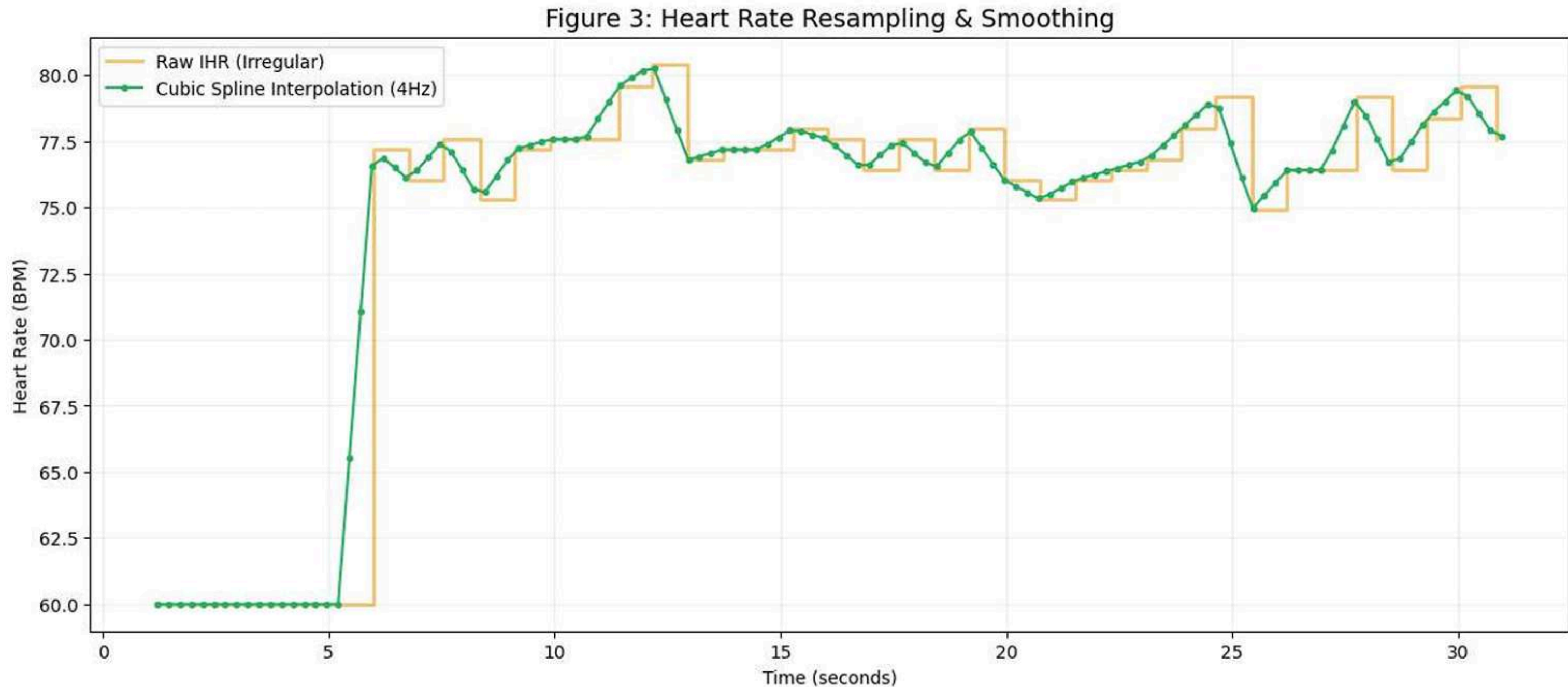
2. HEARTBEAT DETECTION AND RESAMPLING

Located R-peaks, calculated Inter-Beat Intervals (IBI), and handled physiological outliers to create a uniform heart rate signal.

- `mne.preprocessing.find_ecg_events`: For automated R-peak detection.
- `scipy.signal.medfilt`: Applied a **Median Filter** to remove outliers from the IBI sequence.

2.HEARTBEAT DETECTION AND RESAMPLING

- `scipy.interpolate.interp1d`: Performed **Linear Interpolation** to convert irregular heartbeats into a continuous signal sampled at 4Hz.



3. 30 SEC WINDOWING AND NORMALIZATION

Divided the continuous 4Hz signal into 30-second windows (120 samples each) and standardized the data for each window.

- Manual Slicing (for loops): Segmented the signal into 30s epochs to match standard sleep staging protocols.
- `numpy.mean` & `numpy.std`: Applied **Z-score Normalization per window** to ensure the model focuses on morphological patterns rather than amplitude variations.

4. FREQUENCY-DOMAIN FEATURE EXTRACTION

- `scipy.signal.welch`: Calculated the **Power Spectral Density (PSD)** to transform the signal from the time domain to the frequency domain.
- `scipy.integrate.trapezoid`: Performed integration to compute Low Frequency (LF) ,High Frequency (HF) power,Normalized Units (nu) to represent the relative balance of the autonomic nervous system, independent of total power variations.
-

4. ANNOTATION PARSING & LABELING

Extracted sleep stage from XML files and synchronized it with the segmented windows.

- `xml.etree.ElementTree (ET)`: Parsed XML files to extract event concepts, start times, and durations.
- Dictionary Mapping: Converted text labels ('Wake', 'Stage 1',..) into numerical values for the classifier.

5. FINAL FORMATTING

- **numpy.expand_dims:** Adjusted array shapes to be compatible with Temporal Models (adding a channel dimension for CNNs/LSTMs).
- **numpy.save:** Exported the final signals, features, and labels as .npy files for efficient loading during training.

```
... Signals Shape: (12301, 120, 1)
     Features Shape: (12301, 5)
     Labels Shape: (12301,)
     Unique Labels: [0 1 2 3 4]
```

3. DATA BALANCING & AUGMENTATION

FEATURE FUSION

Flattened the 3D signal arrays and horizontally stacked them with the handcrafted HRV features to create a combined feature vector.

- `numpy.reshape`: To flatten the signal dimensions.
- `numpy.hstack`: To combine temporal signals and statistical features into one input matrix.

3. DATA BALANCING & AUGMENTATION

STRATIFIED SPLITTING

- Divided the data into 70% Training, 15% Validation, and 15% Testing. Used stratification to ensure that the percentage of each sleep stage remains consistent across all sets.
- Tools:
- `sklearn.model_selection.train_test_split`: To create the three independent datasets while maintaining class balance using the `stratify` parameter.

3. DATA BALANCING & AUGMENTATION

CLASS BALANCING [SMOTE]

Applied the Synthetic Minority Over-sampling Technique (SMOTE) only to the training set to resolve the imbalance between sleep stages (e.g., more Wake samples than Stage 1).

- `imblearn.over_sampling.SMOTE`: To generate new, synthetic training samples based on the nearest neighbors of the minority classes.

3. DATA BALANCING & AUGMENTATION

SYNTHETIC SIGNAL SMOOTHING

To ensure the newly generated (synthetic) signals are physically plausible, a moving average filter was applied specifically to the samples created by SMOTE.

- `numpy.convolve`: Applied a mathematical convolution with a uniform kernel (`ones(5)/5`) to smooth out any sharp edges or noise in the synthetic ECG signals.

3. DATA BALANCING

TEMPORAL MULTI-INPUT RECONSTRUCTION

Separated the combined data back into signals and features, then concatenated them into a final 3D format suitable for temporal models.

- Custom Python Function (process_back): To accurately slice and reshape the signals (120 samples) and features.
- numpy.concatenate: To merge the 2D features into the signal's time-steps, creating the final input for the Deep Learning model.

3. DATA BALANCING

TEMPORAL MULTI-INPUT RECONSTRUCTION

Separated the combined data back into signals and features, then concatenated them into a final 3D format suitable for temporal models.

- Custom Python Function (process_back): To accurately slice and reshape the signals (120 samples) and features.
- `numpy.concatenate`: To merge the 2D features into the signal's time-steps, creating the final input for the Deep Learning model.

```
[2]
```

```
... Done! Train: 17895, Val: 1845, Test: 1846
```



TEMPORAL WINDOWING

Capturing Sleep Dynamics:

- Sequential Loading: Processes 5 consecutive windows (150 seconds) instead of a single epoch to capture sleep stage transitions.
- Targeting: The model predicts the stage of the last window based on the context of the previous four.
- Input Shape: (Batch, 5, 1, 120) – representing a sequence of 5 IHR signal segments.



HYBRID MODEL ARCHITECTURE [CNN-LSTM]

- Feature Extraction (ResNet-1D): Uses Residual Blocks to extract deep morphological patterns from the IHR signal while preventing gradient vanishing.
- Temporal Learning (Bi-LSTM): A 2-layer Bidirectional LSTM analyzes the sequence of extracted features to understand the "history" of the heart rate.
- Decision Head: A Fully Connected layer with Softmax outputting probabilities for 5 sleep stages (Wake, N1, N2, N3, REM).



TRAINING & OPTIMIZATION

- Optimizer: Adam (LR=0.0005) with Weight Decay to ensure stable learning and prevent overfitting.
- Dynamic Scaling: ReduceLROnPlateau scheduler to decrease learning rate when performance plateaus.
- Checkpointing: Automated saving of the "Best Model" based on peak Validation Accuracy.
- Goal: Achieving high generalization across different sleep cycles and subjects.



TRAINING & OPTIMIZATION

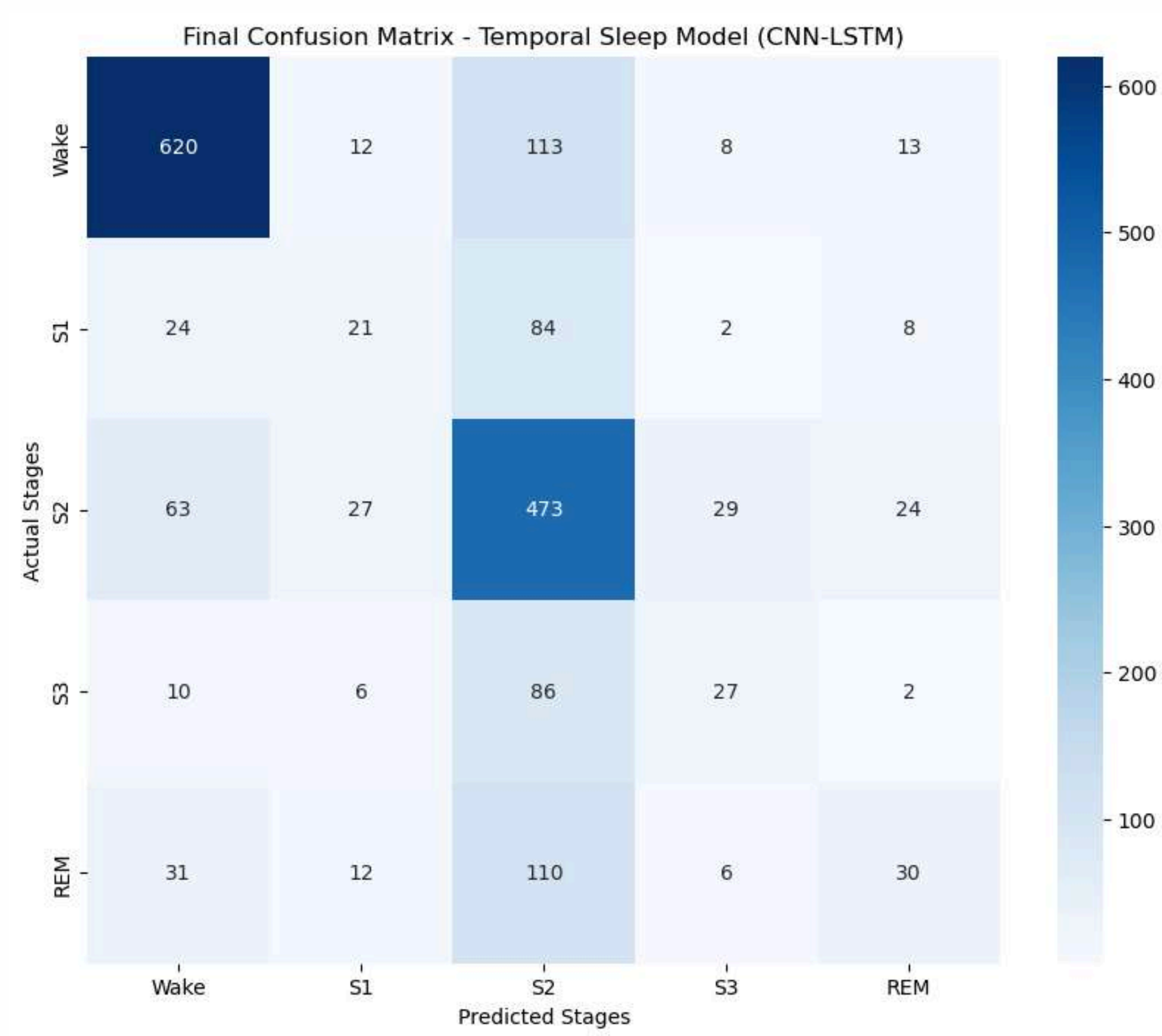
- **Best Validation Accuracy: 63.6%**

```
Epoch [21/40] | Train Acc: 82.05% | Val Acc: 62.05% | LR: 0.000500
Epoch [22/40] | Train Acc: 82.14% | Val Acc: 61.81% | LR: 0.000500
Epoch [23/40] | Train Acc: 82.11% | Val Acc: 52.58% | LR: 0.000250
Epoch [24/40] | Train Acc: 84.62% | Val Acc: 61.98% | LR: 0.000250
...
Epoch [39/40] | Train Acc: 88.51% | Val Acc: 63.06% | LR: 0.000125
Epoch [40/40] | Train Acc: 88.44% | Val Acc: 59.04% | LR: 0.000125

Done! Best Val Accuracy: 63.61%
```

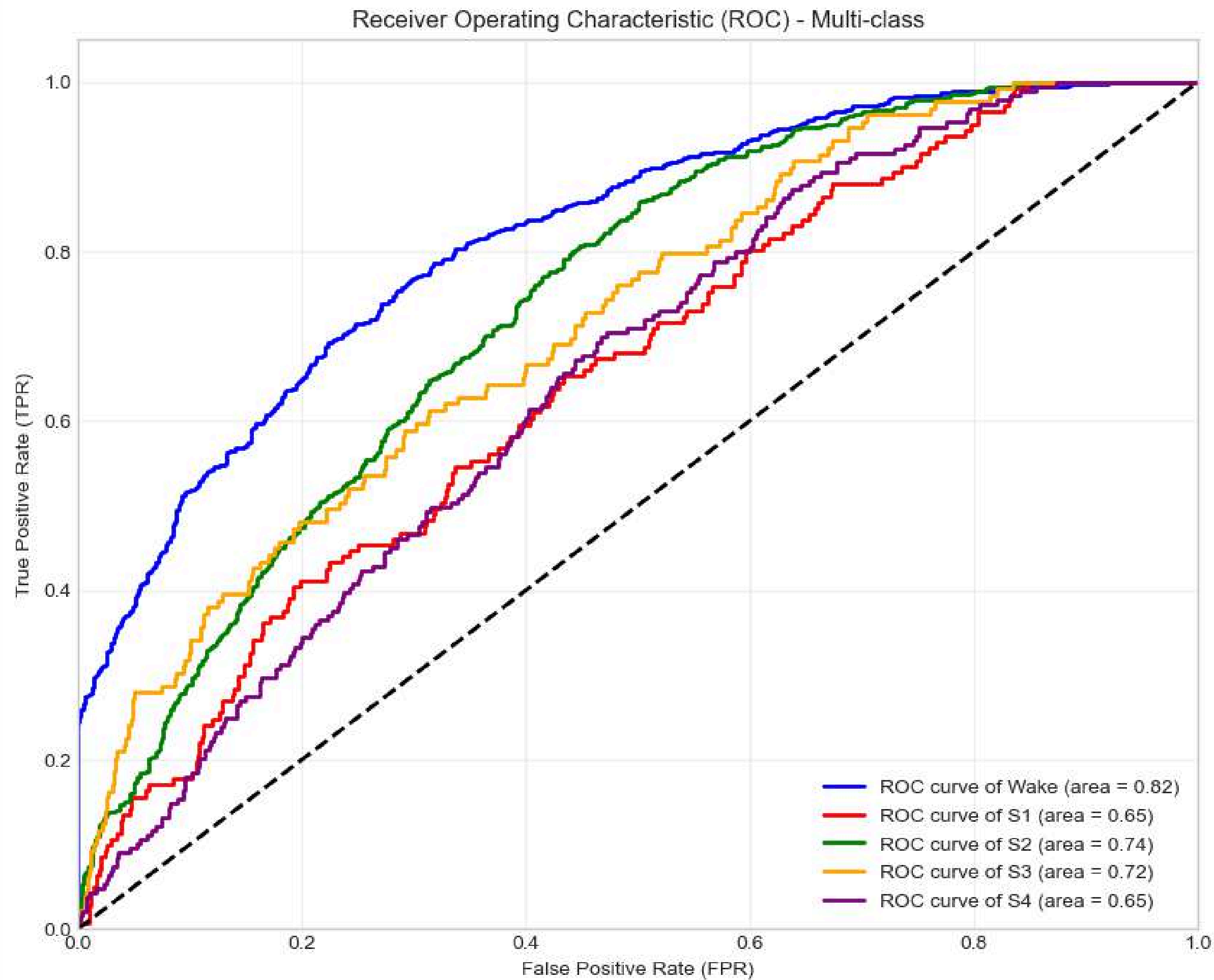
- **Test Accuracy: 63.6%**

```
Final Test Accuracy: 63.61%
```



FINAL CONFUSION MATRIX

ROC CURVE





THANK YOU!

